

● Q1 — ARRAY / SORTING (MOST IMPORTANT)

✦ ARRAY OPERATIONS CORE IDEA

Insert → shift right

Delete → shift left

Reverse → swap

Search → linear scan

🔥 INSERT

```
for(i=count;i>pos;i--){
    arr[i]=arr[i-1];
}
arr[pos]=val;
count++;
```

🔥 DELETE

```
for(i=pos;i<count-1;i++){
    arr[i]=arr[i+1];
}
count--;
```

🔥 SEARCH

```
for(i=0;i<count;i++){
    if(arr[i]==key){
        System.out.println(i);
        break;
    }
}
```

🔥 REVERSE

```
int i=0,j=count-1;
while(i<j){
    int t=arr[i];
    arr[i]=arr[j];
    arr[j]=t;
    i++; j--;
}
```

🔥 BUBBLE SORT (MOST SAFE)

```
for(i=0;i<count-1;i++){
    for(j=0;j<count-1-i;j++){
        if(arr[j]>arr[j+1]){
            int t=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=t;
        }
    }
}
```

🟡 Q2 — STACK / QUEUE

🔥 STACK (LIFO)

top = -1

PUSH

arr[++top]=val;

POP

arr[top--];

CHECK

```
empty = top == -1  
full = top == size-1
```

🔥 QUEUE (FIFO)

```
front = 0  
rear = -1
```

ENQUEUE

```
arr[++rear]=val;
```

DEQUEUE

```
arr[front++];
```

CHECK

```
empty = front > rear  
full = rear == size-1
```

● Q2 OPTION — LINKED LIST (BASIC)

NODE

```
class Node{  
    int data;  
    Node next;  
}
```

ADD NODE

```
if(head==null) head=newNode;  
else go to last and attach
```

DISPLAY

```
temp=head;
while(temp!=null){
    print(temp.data);
    temp=temp.next;
}
```

DELETE BY VALUE

find node → skip it using:
temp.next = temp.next.next;

Q3 — TREE / BST / BFS / DFS

NODE

```
class Node{
    int data;
    Node left,right;
}
```

BST RULE

Left < Root < Right

INSERT BST

if(val<root) go left
else go right

SEARCH BST

```
if(root==null) false
if(root.data==key) true
else go left/right
```

TRAVERSALS

INORDER (MOST IMPORTANT)

Left → Root → Right
(sorted output)

PREORDER

Root → Left → Right

POSTORDER

Left → Right → Root

BFS (LEVEL ORDER)

Use QUEUE
visit level by level

⚡ SUPER SHORT MEMORY TRICKS

Array = shift game

Stack = top pointer

Queue = front/rear

Tree = recursion

BST = ordered tree

BFS = queue

DFS = recursion

● Q1 — ARRAY (FULL PROGRAM CORE FUNCTIONS)

```
// INSERT
for(i=count;i>pos;i--){
    arr[i]=arr[i-1];
}
arr[pos]=val;
count++;

// DELETE
for(i=pos;i<count-1;i++){
    arr[i]=arr[i+1];
}
count--;

// SEARCH
for(i=0;i<count;i++){
    if(arr[i]==key){
        System.out.println(i);
        break;
    }
}

// REVERSE
int i=0,j=count-1;
while(i<j){
    int t=arr[i];
    arr[i]=arr[j];
    arr[j]=t;
    i++; j--;
}
```

🔥 BUBBLE SORT (SAFE Q1 SORT)

```
for(int i=0;i<count-1;i++){
    for(int j=0;j<count-1-i;j++){
```

```

        if(arr[j] > arr[j+1]){
            int t=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=t;
        }
    }
}

```

● Q2 — STACK (FULL CODE)

```

class Stack {
    int arr[];
    int top;
    int size;

    Stack(int size){
        this.size = size;
        arr = new int[size];
        top = -1;
    }

    boolean isFull(){ return top == size-1; }
    boolean isEmpty(){ return top == -1; }

    void push(int val){
        if(isFull()) System.out.println("Full");
        else arr[++top]=val;
    }

    void pop(){
        if(isEmpty()) System.out.println("Empty");
        else System.out.println(arr[top--]);
    }

    void display(){
        for(int i=0;i<=top;i++)
            System.out.print(arr[i]+" ");
    }
}

```

```
}  
}
```

● Q2 — QUEUE (FULL CODE)

```
class Queue {  
    int arr[];  
    int front;  
    int rear;  
    int size;  
  
    Queue(int size){  
        this.size=size;  
        arr=new int[size];  
        front=0;  
        rear=-1;  
    }  
  
    boolean isFull(){ return rear==size-1; }  
    boolean isEmpty(){ return front>rear; }  
  
    void enqueue(int val){  
        if(isFull()) System.out.println("Full");  
        else arr[++rear]=val;  
    }  
  
    void dequeue(){  
        if(isEmpty()) System.out.println("Empty");  
        else System.out.println(arr[front++]);  
    }  
  
    void display(){  
        for(int i=front;i<=rear;i++)  
            System.out.print(arr[i]+" ");  
    }  
}
```

● Q2 — LINKED LIST (BASIC FULL)

```
class Node{
    int data;
    Node next;

    Node(int data){
        this.data=data;
        next=null;
    }
}

class LinkedList{
    Node head;

    void add(int val){
        Node n=new Node(val);

        if(head==null){
            head=n;
        }else{
            Node t=head;
            while(t.next!=null)
                t=t.next;
            t.next=n;
        }
    }

    void display(){
        Node t=head;
        while(t!=null){
            System.out.print(t.data+" ");
            t=t.next;
        }
    }

    void delete(int val){
        if(head==null) return;
```

```
    if(head.data==val){
        head=head.next;
        return;
    }

    Node t=head;
    while(t.next!=null && t.next.data!=val)
        t=t.next;

    if(t.next!=null)
        t.next=t.next.next;
}
}
```

Q3 — TREE / BST / TRAVERSAL (FULL CORE)

NODE

```
class Node{
    int data;
    Node left,right;

    Node(int val){
        data=val;
        left=right=null;
    }
}
```

BST INSERT

```
Node insert(Node root,int val){
    if(root==null)
        return new Node(val);

    if(val < root.data)
```

```
        root.left = insert(root.left,val);
    else
        root.right = insert(root.right,val);

    return root;
}
```

SEARCH

```
boolean search(Node root,int key){
    if(root==null) return false;

    if(root.data==key) return true;

    if(key < root.data)
        return search(root.left,key);
    else
        return search(root.right,key);
}
```

INORDER (MOST IMPORTANT)

```
void inorder(Node root){
    if(root!=null){
        inorder(root.left);
        System.out.print(root.data+" ");
        inorder(root.right);
    }
}
```

PREORDER

```
void preorder(Node root){
    if(root!=null){
        System.out.print(root.data+" ");
        preorder(root.left);
        preorder(root.right);
    }
}
```

```
    }  
}
```

POSTORDER

```
void postorder(Node root){  
    if(root!=null){  
        postorder(root.left);  
        postorder(root.right);  
        System.out.print(root.data+" ");  
    }  
}
```

BFS (LEVEL ORDER)

```
Queue<Node> q = new LinkedList<>();
```

```
void bfs(Node root){  
    if(root==null) return;  
  
    q.add(root);  
  
    while(!q.isEmpty()){  
        Node temp=q.poll();  
  
        System.out.print(temp.data+" ");  
  
        if(temp.left!=null)  
            q.add(temp.left);  
  
        if(temp.right!=null)  
            q.add(temp.right);  
    }  
}
```